



(տարբերակ 4)

Առաջադրանք 1: Ցուցակների ֆիլտրում (List Comprehension-ի և Lambda ֆունկցիայի օգտագործում) (3 միավոր)

Գրել ծրագիր, որը

- ա) օգտատիրոջից կստանա n թիվը և կգեներացնի n երկարությամբ պատահական թվերից կազմված ցուցակ: (1 միավոր)
- բ) կգրի lambda ֆունկցիա, որը թիվը կրկնապատկում է: Կստեղծի `map()` ֆունկցիա, որը ցուցակի յուրաքանչյուր տարր կրկնապատկում է: (1 միավոր)
- գ) կտալի սգբնական և վերջնական ցուցակները: (1 միավոր)

Python

#Մուտք

7

#Ելք

[7, 5, 15, 1, 6, 4, 3] #այս ցուցակի գեներացված է պատահականորեն

[14, 10, 30, 2, 12, 8, 6]

Առաջադրանք 2: Գործողություններ ներդրված ցիկլերում: (4 միավոր)

Գրել ֆունկցիա, որը կստանա 3×3 չափի մատրից (ներդրված ցուցակ), կհաշվի

ա) այն սյունը, որը տարրերի գումարը ամենամեծն է, և կտալի այն

բ) մատրիցի բոլոր բացասական թվերը, և կտալի այն

գ) զույգ (դրական) թվերի գումարը, և կտալի այն:

Python

#Մուտք

[[12, -7, 1],
[14, 3, 16],
[-2, 14, 6],]

#Ելք

"Սյունների տարրերի գումարներից ամենամեծը 24 է:"

"Բացասական տարրերն են -7, -2: "

"Զույգ թվերի գումարը կլինի 62: "

Առաջադրանք 3: Արագ դասակարգման ալգորիթմի կիրառում (Quick Sort) (6 միավոր)

Գրել ֆունկցիա, որը կստանա տողերից կազմված ցուցակ և կդասակարգի այն Quick Sort ալգորիթմով՝ ըստ տողերի երկարության:



(տարբերակ 4)

Այս ալգորիթմի ժամանակ ընտրում է մի «barrier» տարր ցուցակից, տարրերը բաժանում է երեք մասի՝ ավելի փոքր երկարություն ունեցող, ավելի մեծ երկարություն ունեցող, և barrier տարրին հավասար երկարություն ունեցող: Ապա ռեկուրսիվ ձևով տեսակավորում է ավելի փոքր տարրերի և ավելի մեծ տարրերի ցուցակները առանձին-առանձին:

Ուշադրություն!!! Պարտադիր է խնդիրը Qucik Sort ալգորիթմով:

Python

```
#Մուտք
["apple", "new", "all", "sort", "avocado", "or"]
#Ելք
["or", "new", "all", "sort", "apple", "avocado"]
```

Առաջադրանք 4: Օբյեկտ-կողմնորոշված ծրագրավորում (OOP): (7 միավոր)

ա) Ստեղծել **footballer** անունով class, որը կունենա հետևյալը հատկությունները

- name-անուն
- year_ ծննդյան տարեթիվ (private)
- weight - քաշը
- heightl- հասակ (սմ)
- nationality - ազգություն (private)
- current_club- ներկայիս ակումբը
- position- խաղային դիրք (օրինակ՝ goalkeeper)

Մեթոդներ

- age-հաշվում և վերադարձնում է տարիքը
- info- վերադարձնում է անունը տարիքը, ազգությունը և ներկայիս ակումբը
- update_weight (new_weight)- թարմացնում է քաշը դարձնելով new_weight
- change_club (new_club) - փոխում է խաղացողի ակումբը
- bmi - հաշվում է մարմնի զանգվածի ինդեքսը՝
$$bmi = \frac{\text{քաշ (կգ)}}{\text{հասակ}^2 \text{ (մ)}}$$

բ) Ստեղծել ex_footballer ենթադասը, որը կժառանգի footballer դասի, կունենա լրացուցիչ մեթոդ, որը current_club-ը դրանի None: Հիմնական դասից չի ժառանգի change_club մեթոդը: